Table 4: Statistics of the benchmark suite. Numbers indicate the count of **unique prompts** in each split.

Dataset	Scenario	Train	Validation	Test	Total
LongBench	Long-Sequence	330	110	110	550
ZeroSCROLLS	Long-Sequence	330	110	110	550
IFEval	Reasoning	330	110	110	550
CRUXEval	RL	480	160	160	800
GSM8K	RL	4,483	1,494	1,494	7,471
LiveCodeBench	RL	633	211	211	1,055
MATH	RL	4,500	1,500	1,500	7,500
MBPP	RL	1,157	386	386	1,929
MMLU-STEM	RL	1,891	630	630	3,151
Total		**14,134**	**4,711**	**4,711**	**23,556**

As visualized in Figure 4, our benchmark suite is composed of a diverse set of datasets, categorized into three primary scenarios: Long-Sequence (4.7%), Reasoning (2.3%), and Reinforcement Learning (RL) (93.0%). Detailed statistics for each dataset, including the count of unique prompts in the training, validation, and test splits, are provided in Table 4. All datasets are partitioned into training, validation, and test sets following a 3:1:1 ratio.

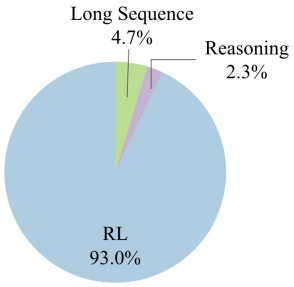


Figure 4: Proportional distribution of unique prompts across the three primary scenarios.

Specifically, the Long-Sequence scenario includes:

- **LongBench** (Bai et al., 2024a) is a bilingual, multi-task benchmark designed to assess the understanding of long text contexts. It encompasses a variety of tasks such as single and multi-document question answering, summarization, and code completion.
- **ZeroSCROLLS** (Shaham et al., 2023) provides a suite of datasets focused on zero-shot evaluation of long-text comprehension. It includes tasks that require synthesizing information across lengthy documents, such as summarization, question answering, and sentiment classification.

The Reasoning scenario contains:

- **IFEval** (Zhou et al., 2023) is a dataset consisting of prompts with explicit and verifiable instructions. It is used to evaluate a model's ability to adhere to constraints and follow complex directives.

The RL scenario includes:

- **CRUXEval** (Gu et al., 2024) is a dataset for evaluating code reasoning, understanding, and execution. It tests a model's ability to predict the output of code snippets and to determine the necessary input to achieve a desired output.
- **LiveCodeBench** (Jain et al., 2025) is a dataset for code generation that features problems from competitive programming websites, focusing on problem-solving with varying levels of difficulty.
- **MBPP** (Austin et al., 2021) is a dataset containing entry-level Python programming problems that can be solved with short, self-contained functions.

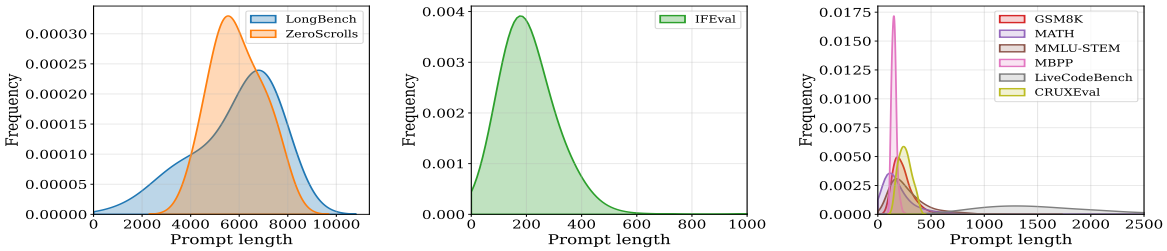
- **GSM8K** (Cobbe et al., 2021) is a dataset of grade school math word problems that require multi-step reasoning to solve.
- **MATH** (Hendrycks et al., 2021b) is a dataset of challenging mathematical problems that require sophisticated reasoning and problem-solving abilities.
- **MMLU-STEM** (Hendrycks et al., 2021a) is a subset of the Massive Multitask Language Understanding benchmark that focuses on science, technology, engineering, and mathematics subjects, designed to test a model's knowledge and problem-solving skills in these areas.

C.2 Prompt and Response Length Distributions

As detailed in Table 5 and visualized in Figure 5 and Figure 6, our benchmark features a wide diversity of sequence lengths. Prompt lengths range from the short, concise queries in reasoning datasets like MATH and GSM8K, which average a few hundred characters, to the extensive contexts in long-sequence datasets like LongBench and ZeroSCROLLS, which average nearly 6,000 characters. Response lengths are similarly varied, reflecting the diverse complexity of the tasks that demand outputs ranging from single-word answers to detailed reasoning and comprehensive code solutions.

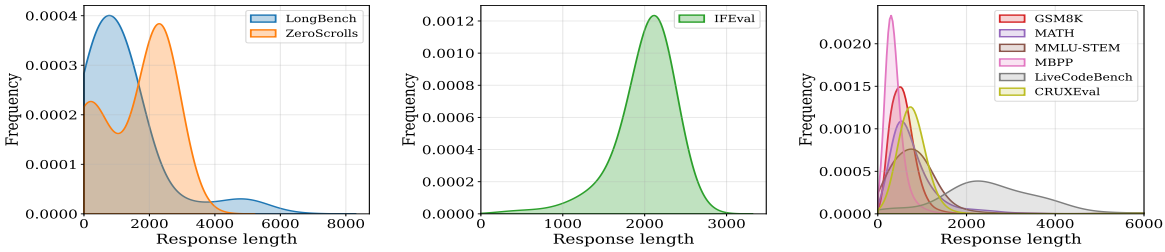
Table 5: Statistical summary of prompt and response lengths across all datasets. The table presents the minimum, maximum, and mean length for each category.

Dataset	Prompt Length			Response Length		
	Min	Max	Mean	Min	Max	Mean
CRUXEval	162	387	257	1	6,841	1,136
LiveCodeBench	572	4,089	1,547	1	7,288	2,188
MMLU-STEM	43	1,574	280	1	7,530	1,058
MBPP	107	319	153	16	6,412	594
MATH	16	4,309	210	2	7,579	736
GSM8K	42	985	235	11	8,063	598
IFEval	53	1,858	211	2	3,730	1,586
LongBench	624	7,994	5,865	3	7,106	1,172
ZeroSCROLLS	4,004	7,983	5,914	1	3,626	1,420



(a) Long-Sequence Scenario (b) Reasoning Scenario (c) RL Scenario

Figure 5: Prompt length distributions across different scenarios.



(a) Long-Sequence Scenario (b) Reasoning Scenario (c) RL Scenario

Figure 6: Response length distributions across different scenarios.

C.3 Benchmark Dataset Examples

To provide a clearer understanding of our benchmark, this section presents representative examples from our three scenarios. The long-sequence example tests the model's ability to synthesize information from extensive contexts. The reasoning example requires multi-step logical deduction. Finally, the RL example is taken directly from our GRPO training data.

An Example of Long-Sequence Scenario

[Prompt]
Question: Who is Renaud II, Count Of Soissons's uncle?

Context:
Passage 1:
John I, Count of Soissons. John became Count of Soissons after the death of his brother Renaud II in 1099. John married Aveline de Pierrefonds, daughter of Nivelon II, Seigneur de Pierrefonds. They had:
- Renaud III, Count of Soissons.
...

Passage 2:
Adelaide, Countess of Soissons. Adelaide was sovereign Countess of Soissons from 1057 until 1105. She was the daughter of Renaud I, Count of Soissons, and his wife, whose name is unknown. Adelaide married William Busac, Count of Eu. William became Count of Soissons in right of his wife.
...

Passage 5:
Renaud II, Count of Soissons. Renaud II became Count of Soissons after his father's disgrace. It is unclear when he assumed the countship, but after his death in 1099, his brother John became the Count of Soissons.
...

[Response]
Renaud II, Count of Soissons' uncle is William Busac, Count of Eu and Soissons.

An Example of Reasoning Scenario

[Prompt]
What is the next number in this series: 1, 4, 7, 11, 17? Please answer with only mathematical notation without any commas.
[Response]
<think>
To find the next number in the series 1, 4, 7, 11, 17, we first need to identify the pattern. We can do this by finding the differences between consecutive terms:
4 - 1 = 3
7 - 4 = 3
11 - 7 = 4
17 - 11 = 6
The differences between consecutive terms are increasing by 1 each time (3, 3, 4, 6). Following this pattern, the next difference should be: 6 + 2 = 8.
To find the next term in the sequence, we add this difference to the last term of the sequence: 17 + 8 = 25
</think>
<answer>
Thus, the next number in the series is 25.
</answer>

An Example of RL Scenario

[Prompt]
Write a function with the following signature: `def first_Factorial_Divisible_Number(x):`
Write a python function to find the first natural number whose factorial is divisible by x.
[Response]
<reasoning>
To find the first natural number whose factorial is divisible by x, I need to write a function which calculates the factorial of the numbers starting from 1 and checks if the resulting factorial is divisible by x. As soon as I find a number with this property, I return that number.
1. Initialize a variable n (starting from 1) to keep track of the current number.
2. Initialize a variable fact (starting from 1) to store the factorial value.
3. Loop through the numbers starting from 1 and calculate the factorial of each number.
4. Check if the factorial is divisible by x.
5. If it is, return the current number.
This approach should yield the first natural number whose factorial is divisible by x.
</reasoning>
<answer>

```
def first_Factorial_Divisible_Number(x):
    n = 1
    fact = 1
    while True:
        if fact % x == 0:
            return n
        n += 1
        fact *= n
```

</answer>